

Research Candidate 2026-07-16-163434

Status: awaiting-eval. **This candidate is not stable guidance. No stable file was edited, no provider call was spent, nothing was committed.**

Scope

This run was triggered by the provider catalog expiring on 2026-07-19 and by a standing obligation: earlier today the repository promoted release 0.4.0, whose central claim — that an independently executed verification loop lifts a coding agent from 77 to 100 on a real proprietary task — rested entirely on **local measurement with no external corroboration**. Three questions framed the run:

1. Is the model catalog still accurate, and what must a refresh change?
2. Does the outside world corroborate, refine or **refute** the architecture we just promoted?
3. What changed on the two platforms we govern (Claude Code, Codex) since the last pass on 2026-07-12?

Method

Three parallel research passes (catalog, external evidence, platform deep dive), each restricted to official and primary sources, followed by an **independent adversarial verifier** that re-fetched the cited pages and was instructed to refuse charitable readings. Every claim carries a source URL, a tier and an access date of 2026-07-16.

The verifier earned its keep: it **downgraded or refuted three of the seven high-consequence claims** it was given, including one of mine. That ratio is the main quality signal of this run.

Claim coverage

18 accepted claims (`claims.json`); 12 rejected with reasons (`rejected-claims.json`); 6 open questions with no adequate source, recorded rather than papered over.

The result that matters: the literature caught two real problems in what we shipped

1. A provenance leak in our own fixture — found because of a tier-1 vendor finding

Cursor measured that **63% of Opus's successful SWE-bench Pro resolutions retrieved the fix rather than derived it**, and that isolation controls (stripping `.git`, proxying egress) cost 14 points of resolve rate ([source](#)). That prompted an audit of our own benchmark.

Good news: no `.git` exists in any agent workspace — the parser is materialised through `git show` into a plain file, so history mining is impossible.

Bad news, and it is ours: `public/sample/build_sample.py` ships inside every agent workspace, and its docstring names the shipped fix's functions (`_clean_drug_name`, `_form_vocab`), the exact Hermes commits, and states that ground truth is the shipped parser's behaviour asserted by `hidden/grade.py`. The 100-scoring solution named its function `_clean_drug_name` — matching the leak.

This leaks the **shape** of the solution, not its implementation or expected outputs. It was present for **all arms equally**, so the 77-vs-100 delta is unbiased — but the absolute scores are inflated relative to a genuinely cold task, and the fixture's claim to be a clean shadow-replay is weakened. Repair and a confirmation run are the top item in `proposed-changes.md`.

2. The strongest methodological critique of our result was published two days ago — and we can answer half of it for free

[Rethinking the Evaluation of Harness Evolution for Agents](#) (submitted 2026-07-14) makes two demands of any harness claim: compare against a **task-level search baseline at matched budget**, and do not evaluate on the same task family you tuned on.

The first demand is **answered from data we already paid for**:

Approach	Score	Tokens
Best-of-3 unscaffolded sampling	77 (all three trials failed identically)	~883k total
One guided verification loop	100	647k–1.21M

At comparable-or-larger spend, three independent samples never find what one guided iteration finds. **The loop's gain is not extra search.** (Best-of-N would additionally need a selection oracle it does not have; here the point is moot — all three samples scored the same.)

The second demand **stands**: we tuned and measured on `medi-ni`. Replication on a second task family is now the largest open item in the program, recorded as an obligation in `eval-plan.json`.

3. Our null replicates independently — but must be scoped precisely

[Compact Constraint Encoding](#) is a near-exact independent replication of our scaffolding null: 11 models, 16 tasks, 830+ invocations, **no statistically significant difference** in constraint satisfaction between verbose and compact encoding (Cliff's $\delta < 0.01$, replicated across two capability tiers), with ~71% fewer constraint tokens. Their conclusion is ours: the benefit is token reduction, not compliance.

But [Scaffold Effects on GAIA](#) shows that varying control-flow **architecture** moves accuracy by up to 28 points within a single model. That is not a contradiction — it is the distinction that makes our story coherent: our *text* changes bought nothing and our *architectural* change (the loop) bought everything. It does mean the claim must always read "prompt-text scaffolding", never "scaffolding". Our published documents already say exactly that.

[PBT-Bench](#) supplies the mechanism: scaffolding gains are capability-dependent and can go **negative** at the frontier — so our null would likely not replicate on weaker models, and we should not claim it does.

Risks the literature raises against our design

- **Saturated visible suite** ([SpecBench](#)): the visible/held-out gap grows 28 points per tenfold increase in code size, and a saturated visible suite is where hacking hides rather than where it is absent. Our 100/100/100 saturates a suite the agent can read. Two defences verified in this run: the **grader is held out** (the agent never sees `hidden/grade.py`), and a **hardcoding probe on the winning solution found zero drug-name literals** — it is a general implementation. Residual risk: the visible checks and the hidden grader exercise the same single sample.
- **Fixed verifiers decay** ([The Verification Horizon](#)): digest-freezing buys integrity *within* a task and forfeits co-evolution *across* model generations. A scope boundary of our design, not a feature.
- **Property checks are a partial net** ([PBT-Bench](#)): agent-derived invariants miss 17–58% of seeded bugs even at best.
- **Three of our mechanisms are externally unreplicated**: harness-executed verification with structured feedback (no study runs the comparison), guidance-on-failure versus guidance-in-prompt (no source), harness-generated evidence (no source). Our no-progress detection is likewise **unvalidated** by any primary source.

Source changes

`source-registry.patch.json`: 2 updates, 9 additions. The material update is that `developers.openai.com/codex/*` now **308-redirects** to `learn.chatgpt.com/docs/*` with restructured paths — our changelog source URL rots quietly. The additions register the tier-1/tier-2 evidence above so future runs recheck it on schedule.

Model catalog

`model-catalog.patch.json`. **No model was released, renamed or deprecated between 2026-07-12 and 2026-07-16** — the refresh due on 2026-07-19 is not a model swap. It fixes two stale source URLs, records availability gates that land inside the next window (Fable 5 excluded under zero data retention; speed: "fast" on Opus 4.7 erroring after **2026-07-24**;

alias resolution differing per platform, which matters because our catalog calls these "provider-current-alias"), and marks `current-frontier-creative` as having no official backing.

One finding is **deliberately blocked**: our automation recommends selector `haiku` together with `effort: low`, and Haiku 4.5 is absent from Anthropic's affirmative effort-support list. The verifier refused this as an argument from omission — the page never says Haiku is unsupported. **One API call settles it**, and the fix differs by outcome (HTTP 400 = broken runs; ignored = merely misleading). No code changes until then.

Platform deltas that affect this repo

- **Codex hermetic runs**: `codex exec --ephemeral --ignore-user-config --ignore-rules` ([source](#)). Our campaigns are Codex-side, so this is the applicable lever — but the verifier established these are **narrower** than Claude Code's `--bare`; they do not skip skills/plugins/MCP/memory.
- `claude --bare` ([source](#)) skips discovery of hooks, skills, plugins, MCP, memory and `CLAUDE.md`, and will become the `-p` default. Caveats: only Bash and file read/edit tools survive; Anthropic-direct auth must be an API key.
- **Codex hook trust gate** ([source](#)): `repo-local .codex/` hooks load only when the project layer is trusted — a governance harness shipping `repo-local` hooks silently no-ops on a fresh CI clone. Our enforcement is Claude-side, so nothing is broken today.
- **One narrow KB error**: we record `[tools] web_search=true` as "off by default". The boolean key still exists (the verifier caught the first researcher overstating this), but the **default is wrong** — the top-level `web_search` enum defaults to `cached`, an OpenAI-maintained index. A benchmark assuming search was off has been getting a cached index.
- **Two silent-corruption defects** were fixed in Claude Code within the last week (`--json-schema` silently unstructured; truncated stream-json dropping `result`). **No campaign of ours is affected** — we run Codex — but it is the same silent-falsification class we hit locally with stale bytecode.

Expected impact

Nothing here changes runtime behaviour. The value is honesty: one fixture defect to repair, one KB line to correct, an expiring catalog refreshed with no model changes, and — most importantly — external evidence attached to claims we had only measured locally, plus two live critiques of our own headline result recorded rather than buried.

Risks and rollback

The chief risk is the fixture leak: until it is repaired and a confirmation run lands, the 100/100/100 must not be published as a clean shadow-replay. Rollback is deletion of this candidate directory; nothing was applied.

Sources

Source	Tier	Accessed
Reward hacking is swamping model intelligence gains (Cursor)	1	2026-07-16
Harness design for long-running application development (Anthropic)	1	2026-07-16
Claude effort parameter	1	2026-07-16
Claude models overview	1	2026-07-16
Claude model deprecations	1	2026-07-16
Claude Code model configuration	1	2026-07-16
Claude Code headless mode	1	2026-07-16
Claude Code permission modes	1	2026-07-16
Claude Code changelog	1	2026-07-16
Codex models	1	2026-07-16
Codex configuration reference	1	2026-07-16

Source	Tier	Accessed
Codex advanced configuration	1	2026-07-16
Codex non-interactive mode	1	2026-07-16
Codex build skills	1	2026-07-16
Codex changelog	1	2026-07-16
Codex hooks	1	2026-07-16
OpenAI API models	1	2026-07-16
OpenAI API deprecations	1	2026-07-16
Rethinking the Evaluation of Harness Evolution for Agents	2	2026-07-16
Compact Constraint Encoding for LLM Code Generation	2	2026-07-16
Is Three the Magic Number? LLM-Based Repair Loops	2	2026-07-16
SpecBench: Measuring Reward Hacking in Long-Horizon Coding Agents	2	2026-07-16
PBT-Bench: Benchmarking AI Agents on Property-Based Testing	2	2026-07-16
The Verification Horizon: No Silver Bullet for Coding Agent Rewards	2	2026-07-16
To Run or Not to Run: Cost-Effectiveness of Code Execution in LLM-Based Program Repair	2	2026-07-16
Scaffold Effects on GAIA: A Controlled Comparison	2	2026-07-16
VeriScale: Adversarial Test-Suite Scaling	2	2026-07-16
Do Coding Agents Deceive Us? Capped Evaluation with Randomized Tests	2	2026-07-16
Trust but Verify! Verification Design for Test-time Scaling	2	2026-07-16
Large Language Models Cannot Self-Correct Reasoning Yet	2	2026-07-16
Input Reduction Enhanced LLM-based Program Repair	2	2026-07-16
Agentic Property-Based Testing	2	2026-07-16

The last three are recorded as **version-sensitive or adjacent** and are not used as support for current guidance; see `rejected-claims.json`. Tier 4–5 leads (practitioner blogs, social) generated no accepted claim in this run: two attributed findings could not be located and are treated as fabricated.